

POO 3 (JAVA ET JAVA AVANCÉ)

Prof. Nisrine DAD

4° Ingénierie Informatique et Réseaux - Semestre I

Ecole Marocaine des Sciences d'Ingénieur

Année Universitaire : 2024/2025

III. CONCEPTS DE LA PROGRAMMATION ORIENTÉE OBJETS (POO) EN JAVA

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- **Objet** : C'est une structure informatique caractérisée par un état et un ensemble d'opérations exécutables par cet objet qui caractérise son comportement.

Objet = état + comportement (opérations)

Exemples:

Fenêtre	
État	position, taille, bordure, titre, couleur,...
Comportement (Opérations)	dessiner, déplacer, agrandir...

Fichier	
État	nom, répertoire, protections, contenu, ...
Comportement (Opérations)	ouvrir, fermer, supprimer,

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- **Classe** : C'est la description d'un ensemble d'objets ayant une structure de données commune et disposant des mêmes méthodes.

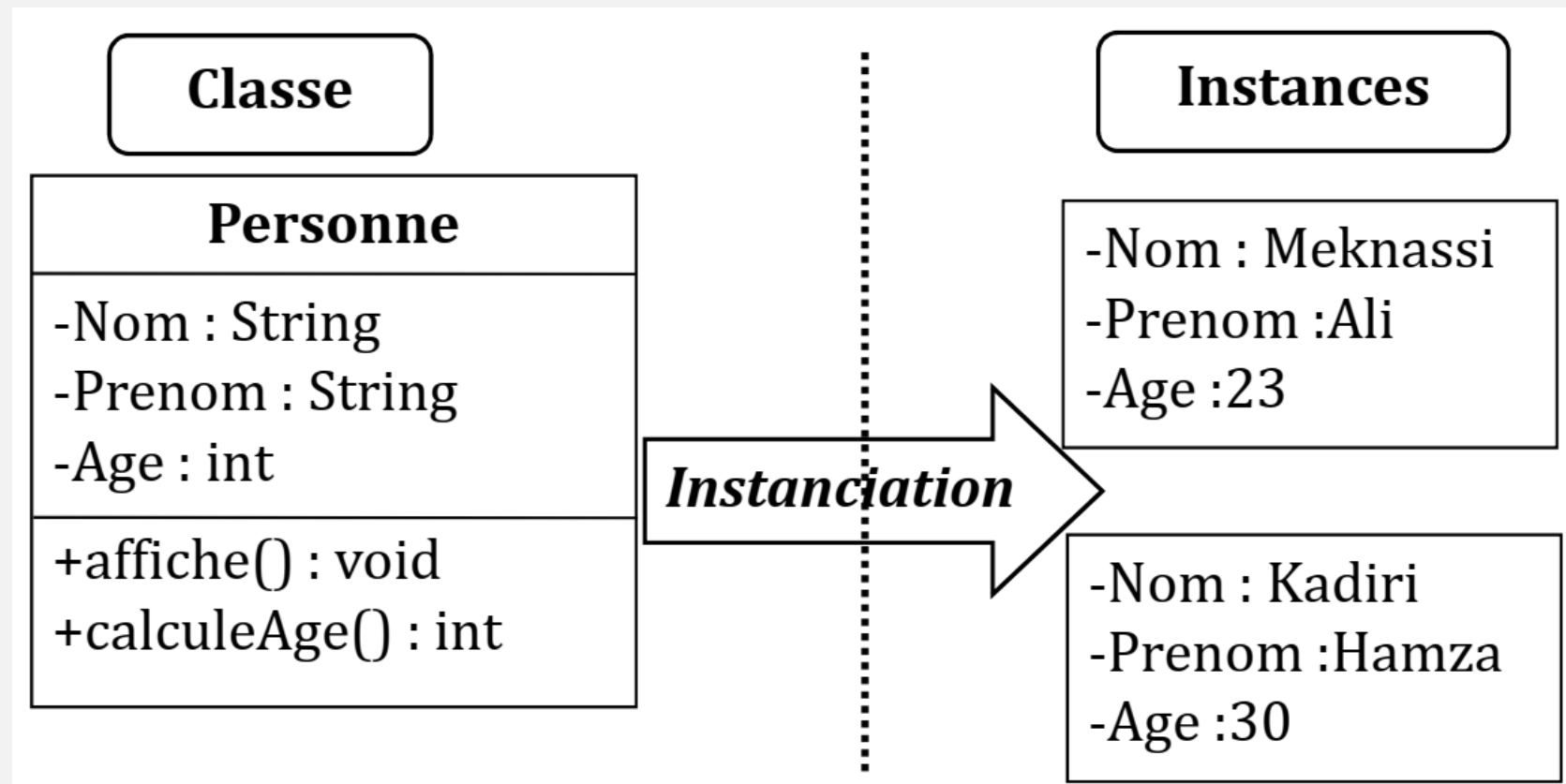
Exemples:

Personne
-Nom : String -Prenom : String -Age : int
+affiche() : void +calculeAge() : int

Materiel
-Nom :String -Prix : double -Nombre : int
+affiche() : void +achete() : void

III. CONCEPTS DE LA POO EN JAVA

Classes et objets



III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- **Classe** : C'est la description d'un ensemble d'objets ayant une structure de données commune et disposant des mêmes méthodes.

Syntaxe de déclaration d'une classe:

```
public class Nom_de_la_classe{  
    type1 p1; // attribut p1  
    type2 p2; //attribut p2  
  
    type3 m3 (...) { // méthode m3 ... }  
    type4 m4 (...) { // méthode m4 ... }  
    ...  
}
```

Les attributs

Les méthodes

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

Exemple d'une classe:

```
public class Point {  
    private int x; // abscisse  
    private int y; // ordonnee  
  
    public void initialise (int abs, int ord){  
        x = abs ;  
        y = ord ;  
    }  
  
    public void deplace(int dx, int dy) {  
        x += dx;  
        y += dy;  
    }  
  
    public void affiche() {  
        System.out.println("Je suis un point de coordonnées " + x + " " + y);  
    }  
}
```

Les attributs

Les méthodes

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- **Méthodes ou fonctions membres:** la définition d'une méthode se compose d'un en-tête et d'un bloc (comme les fonctions ordinaires).

```
public void deplace(int dx, int dy) → L'entête  
{  
    x += dx;  
    y += dy;  
}
```

Le bloc

Remarque : L'ordre des méthodes dans une classe n'a pas d'importance. Par exemple dans la classe **Point** on peut déclarer la méthode **affiche()** puis la méthode **deplace()**, **affiche()** peut être appelée sans problème dans **deplace()**.

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- **Utilisation d'une classe** : Pour pouvoir utiliser une classe il faut d'abord **déclarer un objet**, la déclaration est réalisée de la manière suivante :

NomDeClasse **NomDeVariable** ;

- Exemple:

Point **p1**; /* cette déclaration ne réserve pas d'emplacement pour un objet de type Point, mais seulement un emplacement pour une référence à un objet de type Point.* */

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- Pour **créer une instance** d'une classe, il faut utiliser l'opérateur **new** qui se charge de créer une instance (ou objet) de la classe et de l'associer à la variable.
- L'instanciation est réalisée par la façon suivante :

`NomDeVariable = new NomDeClasse() ;`

- Exemple:

`p1=new Point(); /*crée un emplacement pour un objet de type Point et fournit sa référence en p1.*/`

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

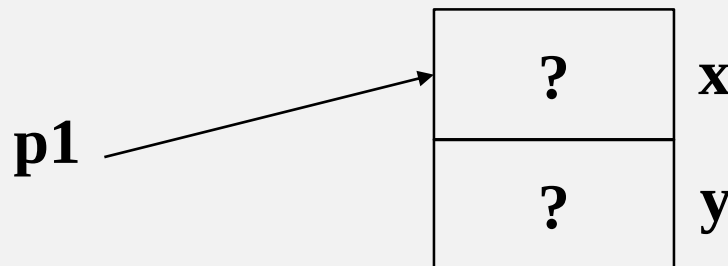
- Il est possible de réunir les deux instructions en une seule :

NomDeClasse **NomDeVariable** = **new** **NomDeClasse**() ;

- Exemple:

Point **p1=new** **Point**(); /*Crée une référence et un emplacement pour un objet de type Point et fournit sa référence en p1.*/*

- La situation peut être schématisée ainsi:



III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- On peut appliquer les méthodes de **Point** à l'objet référencé par **p1**.
- Exemple:

```
public static void main(String[] args) {
```

```
    Point p1=new Point();
```

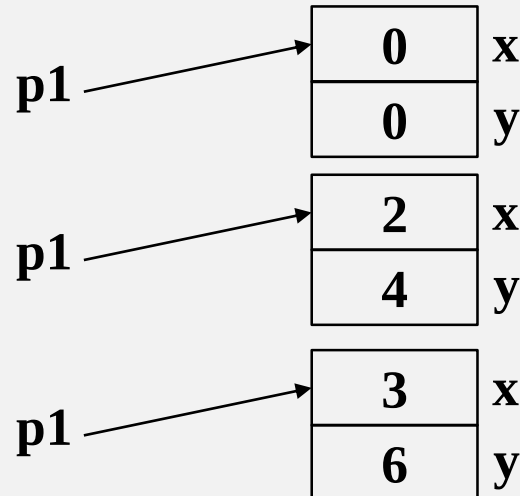
```
    p1.affiche();
```

```
    p1.initialise(2,4);
```

```
    p1.affiche();
```

```
    p1.deplace(1, 2);
```

```
    p1.affiche();
```



```
Je suis un point de coordonnées 0 0
Je suis un point de coordonnées 2 4
Je suis un point de coordonnées 3 6
```

```
}
```

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- **Constructeur de classe** : Un constructeur est une méthode particulière dans une classe. **Il porte le même nom que la classe**, et permet **d'allouer la mémoire** nécessaire à l'objet et **d'initialiser ses attributs**.

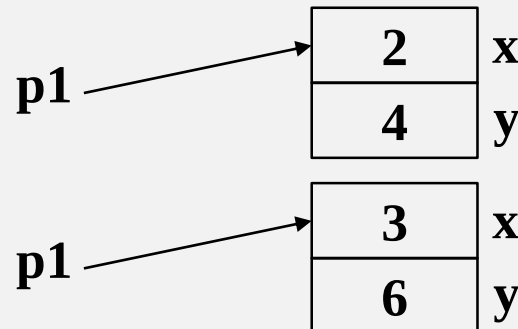
```
public class Point {  
    private int x; // abscisse  
    private int y; // ordonnee  
  
    public Point (int abs, int ord){ // au lieu de la fonction initialise(..)  
        x = abs ; // Le constructeur porte le même nom que la classe et il n'a pas un type de retour  
        y = ord ;  
    }  
    public void deplace(int dx, int dy) {  
        x += dx;  
        y += dy;  
    }  
    public void affiche() {  
        System.out.println("Je suis un point de coordonnées " + x + " " + y);  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- Si on ne crée pas de constructeurs de classes, le constructeur sans arguments est là par défaut.
- Exemple d'utilisation du constructeur:

```
public static void main(String[] args) {  
    Point p1=new Point(2,4);  
    p1.affiche();  
    p1.deplace(1, 2);  
    p1.affiche();  
}
```



Je suis un point de coordonnées 2 4
Je suis un point de coordonnées 3 6

III. CONCEPTS DE LA POO EN JAVA

Classes et objets

- **La destruction d'objets**
- L'un des objectifs du langage Java est d'affranchir le développeur des problématiques de **gestion de mémoire** rencontrées par le C++.
- La gestion de la mémoire a donc été **automatisée**. Ce rôle est délégué au "***Garbage Collector***" ou "**ramasse-miettes**".
- Il libère la mémoire des objets non utilisés, c'est-à-dire qui n'ont plus de référence dans la pile de la mémoire.

III. CONCEPTS DE LA POO EN JAVA

Encapsulation

- L'encapsulation consiste à regrouper les attributs (données) et les méthodes (fonctionnalités) qui manipulent ces données dans une même classe, tout en contrôlant l'accès à ces données.
- Ce contrôle d'accès est rendu possible avec :
 - **private** : visibilité interne à la classe.
 - **public** : visibilité pour toutes les classes.
 - **protected** : visibilité pour les classes du même package **ET** les classes enfants (définies dans n'importe quel package).
 - **default** (aucun des modes ci-dessus n'est spécifié) : visibilité pour les classes du même package.

III. CONCEPTS DE LA POO EN JAVA

Masquage des données

Modifier	Class	Package	Subclass	World
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	✗
<code>no modifier*</code>	✓	✓	✗	✗
<code>private</code>	✓	✗	✗	✗

III. CONCEPTS DE LA POO EN JAVA

Encapsulation: Exemple réel

- Vous avez un compte bancaire.
- Si votre **variable de solde** est déclarée comme variable **publique** dans le logiciel bancaire, n'importe qui peut connaître le solde de votre compte.
- Si votre variable de solde est déclarée privée, personne ne peut voir le solde de votre compte ➔ Votre compte est sécurisé.
- Si vous voulez voir le solde de votre compte, vous devrez accéder uniquement aux membres privés via des méthodes définies dans cette classe.
- Cette méthode demandera votre nom de titulaire de compte ou votre ID utilisateur et votre mot de passe pour l'authentification.

III. CONCEPTS DE LA POO EN JAVA

Getters et setters

- Pour implémenter l'encapsulation dans un programme Java, on doit:
 1. Déclarer les attributs comme **privés** afin qu'ils ne puissent pas être consultés directement par quiconque en dehors de la classe.
 2. Fournir les méthodes **publiques setters** et **getters** dans la classe modifier/récupérer les valeurs des champs (Seulement en cas de besoin).
- Par convention, les accesseurs (**getters**) commencent par **get** et les mutateurs (**setters**) commencent par **set**.
- Faire des accesseurs/mutateurs s'il y a la moindre restriction concernant un champ :
 - Lecture seule (read only): Ne pas définir les setters.
 - Ecriture seule (write only): Ne pas définir les getters.
 - Comportement spécifique (Authentification,...),
 - Possibilité de modification,...

III. CONCEPTS DE LA POO EN JAVA

Getters et setters

```
public class Point {  
    private int x; // abscisse  
    private int y; // ordonnee  
  
    public Point (int abs, int ord){  
        x = abs ;  
        y = ord ;  
    }  
  
    public int getX() {  
        return x;  
    }  
  
    public void setX(int x) {  
        this.x = x;  
    }  
  
    public int getY() {  
        return y;  
    }  
  
    public void setY(int y) {  
        this.y = y;  
    }  
}
```

Ne définir des getters et setters que lorsqu'on en a vraiment besoin dans notre projet → Ils cachent le comportement interne de la classe des classes extérieures.

```
public void setCarColor(String color) {  
    // Seulement la peinture en bleu est autorisée  
    if(color != "bleu" || color != "bleue") {  
        throw new IllegalArgumentException();  
    }  
    this.color = color;  
}
```

Exemple de bonne utilisation d'un setter

III. CONCEPTS DE LA POO EN JAVA

Le mot clé *this*

- On utilise le mot clé *this* dans les circonstances suivantes :
 - 1- Dans une méthode, *this* permet de **référencer l'instance de l'objet en cours d'utilisation**. C'est un objet qui est égale à l'instance de l'objet dans lequel il est utilisé.
 - 2- Dans une méthode, *this* permet **d'éviter les conflits des noms** lorsque les noms d'arguments d'une méthode et les variables locales sont identiques à des noms d'attributs d'une classe.
 - 3- Dans les classes définissant plusieurs constructeurs, un constructeur peut appeler un autre constructeur de cette classe par le mot clé *this()*.

III. CONCEPTS DE LA POO EN JAVA

```
public class Point {  
    private int x; // abscisse  
    private int y; // ordonnee  
    public Point (int abs, int ord){  
        x = abs ; y = ord ;  
    }  
    public Point() {  
        // TODO Auto-generated constructor stub  
    }  
    public Point doubleCoordonnees (){  
        Point resultat=new Point() ;  
        resultat.x=2*this.x ;  
        resultat.y= 2*this.y ;  
        return resultat;  
    }  
    public void affiche() {  
        System.out.println("Je suis un point de  
        coordonnées " + x + " " + y);  
    }  
}
```

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Point p1=new Point(2,4);  
        p1.affiche();  
        Point p1Double=p1.doubleCoordonnees();  
        p1Double.affiche();  
  
    }  
}
```

```
Je suis un point de coordonnées 2 4  
Je suis un point de coordonnées 4 8
```

III. CONCEPTS DE LA POO EN JAVA

```
public class Point {  
    private int x; // abscisse  
    private int y; // ordonnee  
  
    public Point (int x, int y){  
        this.x = x ;  
        this.y = y ;  
        /*this.x fait référence à la variable  
        membre x alors que x fait référence  
        à la valeur de l'argument.*/  
    }  
  
    public void affiche() {  
        System.out.println("Je suis un point de  
        coordonnées " + x + " " + y);  
    }  
}
```

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Point p1=new Point(2,4);  
        p1.affiche();  
  
    }  
}
```

Je suis un point de coordonnées 2 4

III. CONCEPTS DE LA POO EN JAVA

```
public class Point {  
    private int x; // abscisse  
    private int y; // ordonnee  
  
    public Point (int x, int y){  
        this.x = x ;  
        this.y = y ;  
        System.out.println("Constructeur avec  
2 arguments "+x+" et "+y);  
    }  
  
    public Point() {  
        this(0,0);  
        System.out.println("Constructeur sans  
argument");  
    }  
}
```

```
public class Test {  
  
    public static void main(String[] args) {  
  
        Point p0=new Point();  
        System.out.println();  
        Point p1=new Point(2,4);  
  
    }  
}
```

Constructeur avec 2 arguments 0 et 0
Constructeur sans argument

Constructeur avec 2 arguments 2 et 4

III. CONCEPTS DE LA POO EN JAVA

Surcharge des méthodes

- La surcharge d'une méthode permet de définir plusieurs fois une même méthode avec des arguments différents (en type et en nombre).
- Lors de l'appel de la méthode, le compilateur choisit la méthode qui doit être appelée en fonction du nombre et du type des arguments.

```
public void deplace(int dx, int dy) {  
    x += dx;  
    y += dy;  
    System.out.println("deplace(int dx, int dy)");}
```

```
public void deplace (int dx) {  
    x += dx ;  
    System.out.println("deplace(int dx)");}
```

```
public void deplace (short dx) {  
    x += dx ;  
    System.out.println("deplace(short dx)");}
```

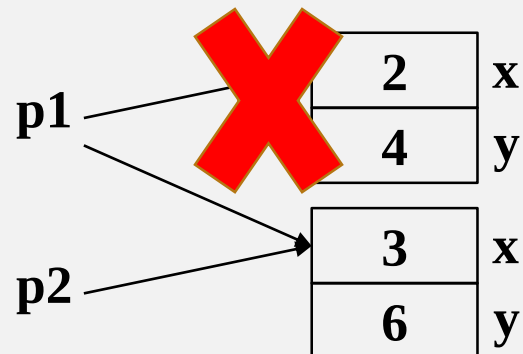
```
Point a = new Point (1, 2) ;  
a.deplace (1, 3);  
a.deplace (2) ;  
short p = 3 ;  
a.deplace (p) ;  
byte b = 2 ;  
a.deplace (b) ;
```

```
deplace(int dx, int dy)  
deplace(int dx)  
deplace(short dx)  
deplace(short dx)
```

III. CONCEPTS DE LA POO EN JAVA

Affectation d'objets

```
public static void main(String[] args) {  
    Point p1=new Point(2,4);  
    Point p2=new Point(3,6);  
    p1= p2;  
}
```



III. CONCEPTS DE LA POO EN JAVA

Exercice

Exercice 1 : équations second degré

- On veut écrire un programme permettant de résoudre les équations du second degré à coefficients réels.
1. Réaliser une classe Equation qui contient les attributs suivants :
 - a : Coefficient du terme en X²
 - b: Coefficient du terme en X¹
 - c : Coefficient du terme en X⁰
 - delta : discriminant
 - r1 : Première racine
 - r2 : Seconde racine

Une équation du second degré à une inconnue x s'écrit sous la forme:

$$a.x^2 + bx + c = 0$$

a, b et c sont des nombres quelconques.

Pour une équation du type $a.x^2 + bx + c = 0$, on calcule le **discriminant Δ** par la formule **$\Delta = b^2 - 4ac$**

→ Si $\Delta < 0$, alors **il n'y a pas de solution.**

→ Si $\Delta = 0$, il y a **une solution double:**

$$x_2 = \frac{-b}{2a}$$

→ Si $\Delta > 0$, il y a **deux solutions distinctes :**

$$x_1 = \frac{-b - \sqrt{\Delta}}{2a} \quad x_2 = \frac{-b + \sqrt{\Delta}}{2a}$$

III. CONCEPTS DE LA POO EN JAVA

Exercice

Exercice 1 : équations second degré

2. Définir un constructeur qui permet d'initialiser les coefficients.
3. Ajouter la méthode résolution pour résoudre l'équation.
4. Ajouter la méthode affichage pour afficher les solutions
5. Écrire un programme TestEquation qui permet de tester la classe Equation et résoudre les équations suivantes:

$$(a) 2x^2 - x - 6 = 0 \quad (b) 2x^2 - 3x + \frac{9}{8} = 0 \quad (c) x^2 + 3x + 10 = 0$$

$$(a) r_1 = -\frac{3}{2} \quad r_2 = 2 \quad (b) r_1 = \frac{3}{4} \quad (c) \text{Pas de solution dans } \mathbb{R}$$

III. CONCEPTS DE LA POO EN JAVA

Exercice

```
public class Equation {  
    private float a, b, c, delta, r1, r2;  
    public Equation(float a, float b, float c) {  
        this.a = a;  
        this.b = b;  
        this.c = c;  
    }  
    public void resolution() {  
        delta = ((b * b) - (4 * a * c));  
        if (delta > 0) {  
            r1 = (float) ((-b - Math.sqrt(delta)) / (2 * a));  
            r2 = (float) ((-b + Math.sqrt(delta)) / (2 * a));  
        }  
        if (delta == 0)  
            r1 = -b / (2 * a);  
    }  
}
```

```
public void affichage() {  
    if (delta > 0)  
        System.out.println( "L'Equation a  
deux solutions possible : " + "  
Racine_1 = " + r1 + " ; " + "  
Racine_2 = " + r2);  
  
    if (delta == 0)  
        System.out.println("L'Equation a une  
seule solution : " + " Racine_1 = "  
+ r1);  
    else  
        System.out.println("L'Equation n'a  
pas de solution !");  
}
```

III. CONCEPTS DE LA POO EN JAVA

Exercice

```
public class Test {  
    public static void main(String[] args) {  
  
        Equation eq1=new Equation(2f,-1f,-6f);  
        eq1.resolution();  
        eq1.affichage();  
  
        Equation eq2=new Equation(2f,-3f,9f/8f);  
        eq2.resolution();  
        eq2.affichage();  
  
        Equation eq3=new Equation(1f,3f,10f);  
        eq3.resolution();  
        eq3.affichage();  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

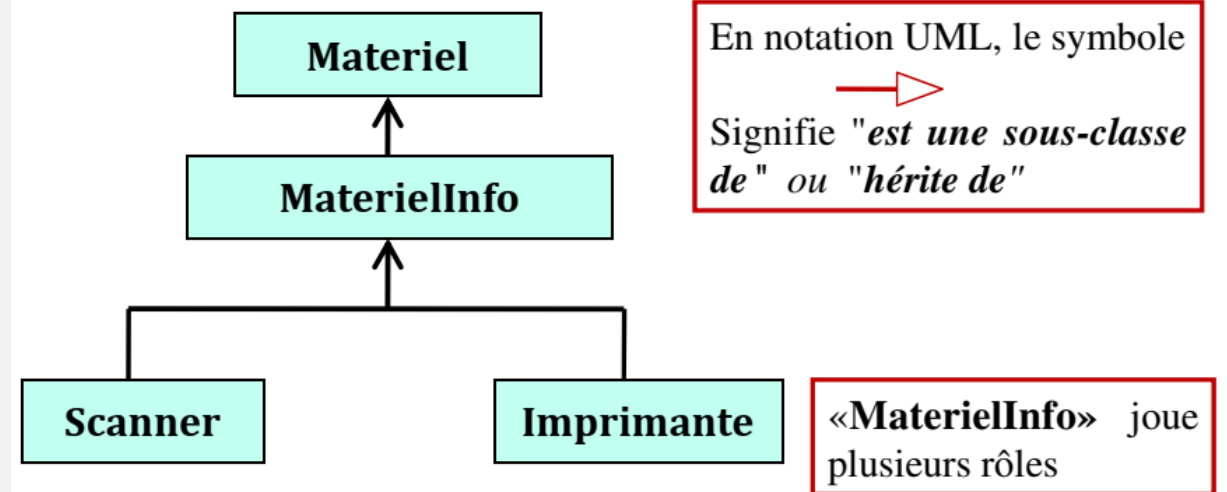
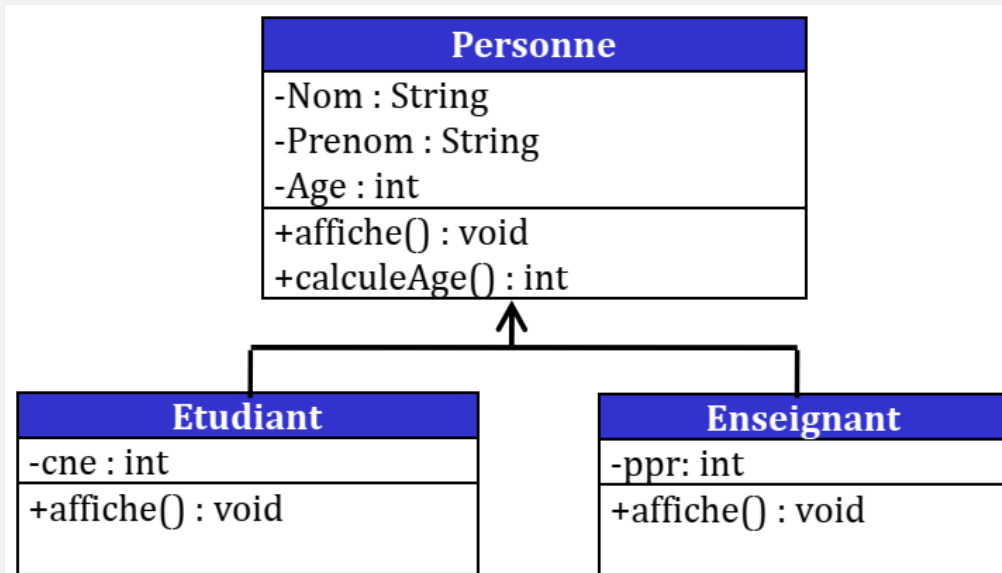
Héritage

- L'héritage est une propriété importante de la programmation orientée objet Java.
- Ce concept permettant d'ajouter des nouvelles fonctionnalités à une classe à partir de la création d'une sous-classe héritant des propriétés de la classe de base et à laquelle s'ajoutent des nouvelles propriétés.
- La classe héritée s'appelle sous-classe (classe dérivée) de la classe mère (super-classe).
- L'héritage en Java, ne permet à une classe d'hériter que d'une seule classe de base, on parle d'héritage simple. En effet, **pas d'héritage multiple**.
- Toutes les classes Java dérivent automatiquement d'une seule et même classe ancêtre : **la classe Object**.
- Donc, l'héritage permet de définir une nouvelle classe à partir d'une classe existante, à laquelle on ajoute de nouvelles données et méthodes.

III. CONCEPTS DE LA POO EN JAVA

Héritage

- L'héritage induit une relation d'arborescence entre les classes.

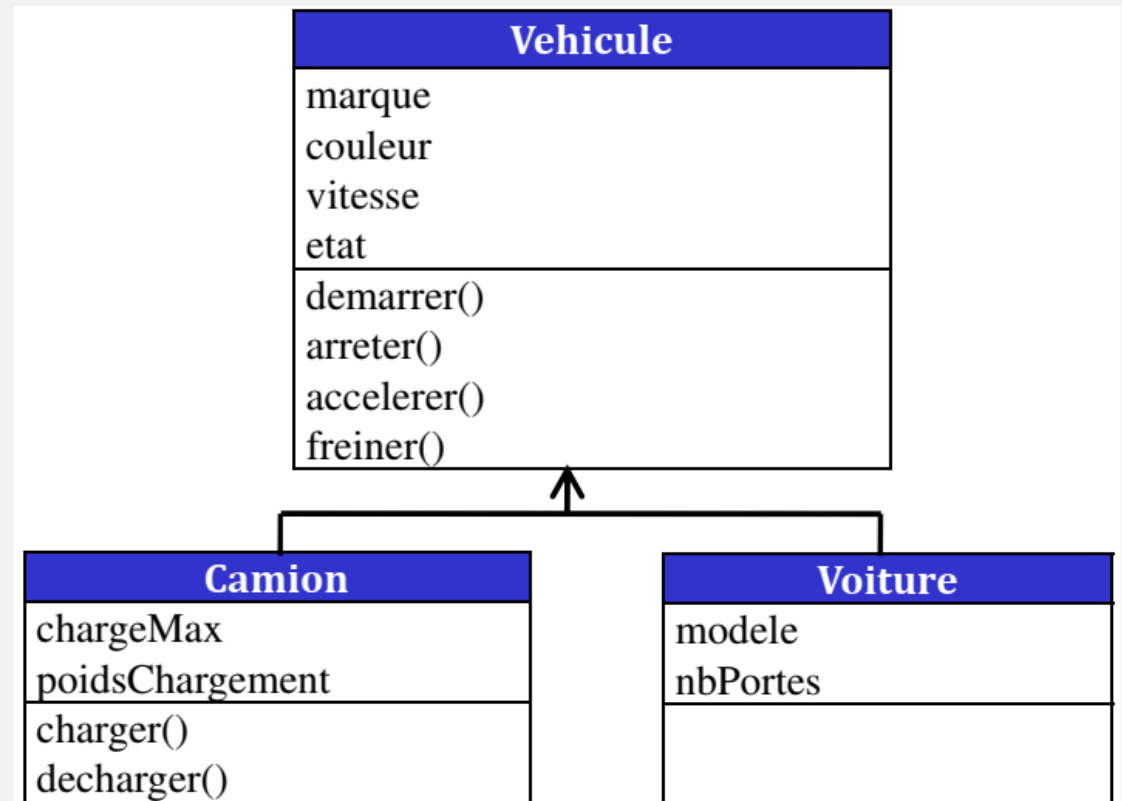


- La classe «**Matériel**» est la classe mère de «**MatérielInfo**».
- La classe «**MatérielInfo**» est la classe mère de «**Scanner**» et «**Imprimante**».
- La classe «**MatérielInfo**» est une sous-classe de «**Matériel**».
- «**Scanner**» et «**Imprimante**» sont des sous-classes de

III. CONCEPTS DE LA POO EN JAVA

Héritage: Exemple

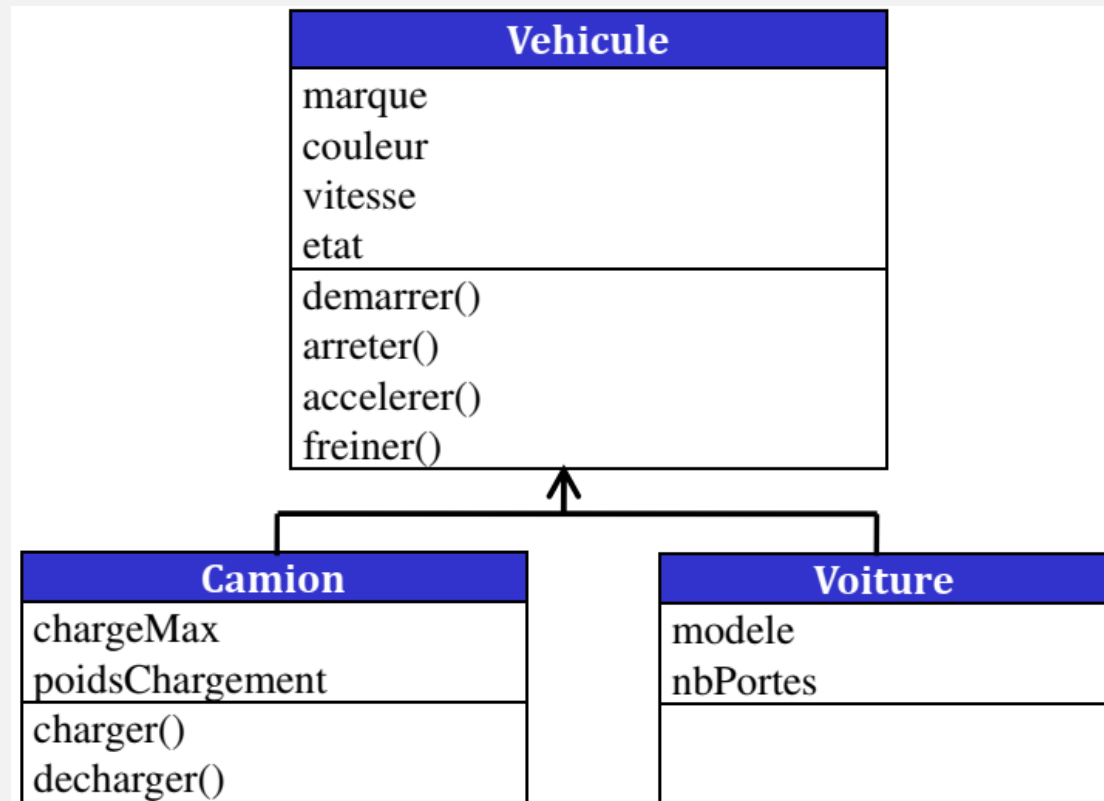
- A partir de la classe **Vehicule**, de nouvelles classes « **Voiture** » et « **Camion** » peuvent être créées, en utilisant l'héritage.
- Elles la spécialisent en y ajoutant de nouvelles propriétés.
- Un tel diagramme, dans le langage UML, est appelé "Diagramme de classes". Il montre les relations entre les classes d'une application.



III. CONCEPTS DE LA POO EN JAVA

Héritage: Exemple

- La classe **Camion** est une classe **dérivée** de **Vehicule**. Elle **hérite** tous ses méthodes et attributs. Elle **étend** la classe mère **Vehicule** en lui ajoutant deux **nouveaux attributs** *chargeMax* et *poidsChargement*, ainsi que deux **nouvelles méthodes** *charger()* et *decharger()*.
- La classe **Voiture** est une classe **dérivée** de **Vehicule** et **hérite** toutes les méthodes et les attributs de Vehicule en lui **ajoutant** deux nouveaux attributs *modele* et *nbPortes*.



III. CONCEPTS DE LA POO EN JAVA

marque
couleur
vitesse
etat

demarrer()
arreter()
accellerer()
freiner()

```
public class Vehicule {  
    protected String marque, couleur;  
    protected double vitesse; // Vitesse actuelle  
    protected int etat; // 0:arrêt, 1:marche, ...  
  
    public Vehicule  
        (String marque, String couleur){  
        this.marque = marque;  
        this.couleur = couleur;  
        vitesse = 0.0;  
        etat = 0;}  
}
```

- Les attributs sont indiqués **protected** pour permettre aux classes filles d'y accéder.

```
    public void demarrer() {  
        etat = 1;}  
  
    public void arreter() {  
        if (vitesse == 0) etat = 0;}  
  
    public void accellerer() {  
        if (etat == 1) vitesse += 5.0;}  
  
    public void freiner() {  
        if (vitesse >= 5.0) vitesse -= 5.0;  
        else vitesse = 0.0;}  
}
```

III. CONCEPTS DE LA POO EN JAVA

Héritage: Exemple

- La déclaration d'une classe dérivée se fait en utilisant le mot clé «**extends**» suivi du nom de la classe de base.
- Un constructeur d'une classe dérivée **doit appeler** un constructeur de sa classe de base. Il suffit de transmettre les paramètres du constructeur de la classe fille au constructeur de la classe mère. Ceci est fait **explicitement** par le mot-clé **super**.

```
public class Voiture extends Vehicule{  
  
    private String modele; // Nouvel attribut  
    private int nbPortes; // Nouvel attribut  
  
    public Voiture(String marque, String modele, String couleur, int nbPortes){  
        super(marque, couleur); // Appel au constructeur de la classe mère  
        this.modele = modele;  
        this.nbPortes = nbPortes;}}}
```

III. CONCEPTS DE LA POO EN JAVA

Le mot clé *super*

1. Le mot-clé *super()* avec les constructeurs.
 - *super()* **devrait être** la première déclaration d'un constructeur (Appel explicite).
 - Si *super()* n'est pas la première déclaration, le compilateur Java appelle implicitement le constructeur *super()* sans arguments.

```
public class SuperBase {  
    String message;  
  
    public SuperBase(String message) {  
        this.message=message;  
    }  
}
```

```
public class SuperSub extends SuperBase {  
  
    public SuperSub(String message) {  
        super(message);  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

Le mot clé *super*

2. Accès aux variables de classe parent

- Nous pouvons accéder à la variable parent à partir de la classe enfant en utilisant le mot clé *super*.

```
public class SuperBase {  
    String message = "super class";  
  
    public SuperBase(String message) {  
        this.message=message;  
    }  
}
```

```
public class SuperSub extends SuperBase {  
    String message = "child class";  
  
    public void getParentMessage() {  
        System.out.println(super.message);  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

Le mot clé *super*

3. Le mot-clé **super** avec remplacement de méthode (Redéfinition de méthode).

```
public class SuperBase {  
    String message = "super class";  
  
    public void printMessage() {  
        System.out.println(message);  
    }  
}
```

```
public class SuperSub extends SuperBase {  
    String message = "child class";  
  
    public SuperSub() {  
        super.printMessage();  
        printMessage();  
    }  
  
    public void printMessage() {  
        System.out.println(message);  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

Héritage: Exercice

- Quelles sont les erreurs commises?
- Corrigez-les.

```
public class Point {  
    private int x; // abscisse  
    private int y; // ordonnee  
  
    public Point (int x, int y){  
        this.x = x ;  
        this.y = y ; }  
  
    public void affiche() {  
        System.out.println("Je suis un  
point de coordonnées " + x + " "  
+ y);}  
}
```

```
public class PointCol extends Point{  
    private String couleur ;  
  
    public PointCol(int abs, int ord,  
        String couleur) {  
        this.x=abs;  
        this.y=ord;  
        this.couleur=couleur;  
    }  
    public void setCouleur (String c) {  
        if(c=="rouge")  
            couleur = c ;  
    }  
    public String getCouleur() {  
        return couleur;  
    }  
}
```


III. CONCEPTS DE LA POO EN JAVA

Héritage: Exercice

- Corrigé

```
public class Point {  
    private protected int x; // abscisse  
    private protected int y; // ordonnee  
  
    public Point (int x, int y){  
        this.x = x ;  
        this.y = y ; }  
  
    public void affiche() {  
        System.out.println("Je suis un  
        point de coordonnées " + x + " "  
        + y);  
    }  
}
```

```
public class PointCol extends Point{  
    private String couleur ;  
  
    public PointCol(int abs, int ord,  
        String couleur) {  
        this.x=abs; // Erreur: x est privé  
        this.y=ord; // Erreur: y est privé  
        super(abs,ord); // On doit commencer par  
        // appeler le constructeur de la classe mère Point  
  
        this.couleur=couleur;  
    }  
    public void setCouleur (String c) {  
        if(c=="rouge")  
            couleur = c ;  
    }  
    public String getCouleur() {  
        return couleur;}  
}
```

III. CONCEPTS DE LA POO EN JAVA

Polymorphisme

- Le **polymorphisme** permet d'utiliser l'héritage comme un mécanisme d'extension en **adaptant le comportement des objets (Redéfinition des méthodes)**.
- On appelle **redéfinition d'une méthode**, la possibilité de définir le comportement d'une méthode selon le type d'objet l'invoquant, c.-à-d., de donner une nouvelle implémentation à une méthode héritée sans changer sa signature.
- Si une méthode est redéfinie dans une classe fille, le **type de retour** doit être **identique** à celui de la méthode correspondante de la classe mère.
- En redéfinissant une méthode, il est possible d'**étendre** sa **zone de visibilité** (private →package →protected →public), mais **non de la restreindre**.

III. CONCEPTS DE LA POO EN JAVA

Polymorphisme: Exemple

- Le polymorphisme permet d'utiliser l'héritage comme un mécanisme d'extension en adaptant le comportement des objets (Redéfinition des méthodes).

```
public class Vehicule {  
    protected String marque,couleur;  
    protected double vitesse;  
    protected int etat;  
  
    public void afficher() {  
        String etatS;  
        if(etat==1)    etatS="marche";  
        else    etatS="arret";  
        System.out.println("Je suis un véhicule  
de marque: "+marque+",de couleur: "+couleur+  
", ma vitesse est : "+vitesse+" Km/h et je  
suis en état de: "+etatS);}}}
```

```
public class Voiture extends Vehicule{  
    private String modele;  
    private int nbPortes;  
  
    public void afficher() {  
        super.afficher();  
        System.out.println("Et je suis aussi une  
voiture de modèle: "+modele+" et j'ai  
        "+nbPortes+" portes.");}}}
```

Dans la classe fille **Voiture**, on a donné à la méthode **afficher()** une nouvelle implémentation sans changer sa signature.

III. CONCEPTS DE LA POO EN JAVA

Polymorphisme: Exemple

- Le comportement de la méthode *afficher()* est défini selon le type d'objet l'invoquant, si c'est **Vehicule** ou **Voiture**.

```
public static void main(String[] args) {  
  
    Vehicule ve1=new Vehicule("Dacia", "Blanc");  
    Camion ca1=new Camion("Honda","Noir",120);  
    Voiture vo1=new Voiture("Dacia", "Noir", "Stepway",4);  
  
    System.out.println("--- Affichage vehicule ----");  
    ve1.afficher(); //La méthode afficher() de Vehicule est  
                   //appelée  
  
    System.out.println("\n--- Affichage voiture ----");  
    vo1.afficher(); //La méthode afficher() de Voiture est  
                   //appelée  
}
```

```
--- Affichage vehicule ----  
- Je suis un véhicule de  
marque: Dacia, de couleur:  
Blanc, ma vitesse est : 0.0  
Km/h et je suis en état de:  
arrêt
```

```
--- Affichage voiture ----  
- Je suis un véhicule de  
marque: Dacia, de couleur:  
Noir, ma vitesse est : 0.0 Km/h  
et je suis en état de: arrêt  
- Et je suis aussi une voiture  
de modèle: Stepway et j'ai 4  
portes.
```

III. CONCEPTS DE LA POO EN JAVA

Polymorphisme: Exemple

- En redéfinissant une méthode *demarrer()* dans **Voiture**, on a étendu sa **zone de visibilité** de **protected** → **public**.
- Les 2 méthodes *demarrer()* ont le même type de retour.

```
public class Vehicule {  
    protected void demarrer() {  
        .....  
    }  
}
```

```
public class Voiture extends Vehicule {  
    public void demarrer() {  
        //1. La méthode est redéfinie et doit avoir le  
        même type de retour que la méthode de base.  
        //2. La redéfinition est avec extension des droits  
        d'accès de protected à public  
        .....  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

Polymorphisme

- C'est toujours la méthode associée au type effectif de l'objet référencé qui est exécutée, même si l'objet est enregistré dans une variable déclarée avec le type d'une classe mère :

```
public static void main(String[] args) {  
  
    Voiture vo2=new Voiture("Renault", "Noir", "Clio 3", 4);  
    Vehicule ve2=vo2;  
    System.out.println("\n--- Affichage voiture ou vehicule? ----");  
    ve2.afficher();  
    //La méthode afficher() de Voiture est appelée  
}
```

```
--- Affichage voiture ou vehicule? ----  
- Je suis un véhicule de marque: Renault, de couleur: Noir, ma vitesse est : 0.0  
Km/h et je suis en état de: arrêt  
- Et je suis aussi une voiture de modèle: Clio 3 et j'ai 4 portes.
```

III. CONCEPTS DE LA POO EN JAVA

Polymorphisme

- Une méthode redéfinie **ne peut pas générer plus d'exceptions contrôlées** que celles qui sont déclarées dans la méthode de la classe parente (mais elle peut en déclarer moins).
- Dans les méthodes de classe (**static**), il n'y a pas de notion de redéfinition, mais la notion de définition.
- **Éviter de :**
 - Surcharger une méthode qui est redéfinie
 - Redéfinir une partie des méthodes surchargées
- Car cela provoque des risques de confusion et crée des pièges lors des modifications ultérieures des classes.

III. CONCEPTS DE LA POO EN JAVA

L'opérateur instanceof

- L'opérateur *instanceof* permet de vérifier si une référence d'un objet est une instance d'une certaine classe ou interface. **Remarque:** On peut utiliser aussi la méthode *getClass()*.

```
Voiture vo2=new Voiture("Renault", "Noir", "Clio 3", 4);  
Vehicule ve2=vo2;
```

```
if(ve2 instanceof Voiture)  
    System.out.println("Voiture");  
else  
    System.out.println("Vehicule");
```

```
Voiture vo2=new Voiture("Renault", "Noir", "Clio 3", 4);  
Vehicule ve2=vo2;  
  
if(ve2.getClass()==Voiture.class)  
    System.out.println("Voiture");  
else  
    System.out.println("Vehicule");
```


III. CONCEPTS DE LA POO EN JAVA

Transtypage: Types de base

Le type visé a un plus **petit** domaine que le type d'origine.

explicite

implicite

```
// Affectation d'un  
//float à un int  
float a=5.2345f;  
int b= (int) a;  
System.out.println(b);
```

```
// Affectation d'un  
//float à un int  
float a=5.2345f;  
int b= a;  
//Erreur compilation  
System.out.println(b);
```

Ce type de casting est interdit en Java

Le type visé a un plus **grand** domaine que le type d'origine.

explicite

implicite

```
// Affectation d'un  
//int à un float  
int a=5;  
float b= (float) a;  
System.out.println(b);
```

```
// Affectation d'un  
//int à un float  
int a=5;  
float b= a;  
System.out.println(b);
```

5.0

5.0

III. CONCEPTS DE LA POO EN JAVA

Transtypage: Types objets

Upcasting

Affecter une valeur qui correspond à une référence d'une instance d'une classe fille, dans une référence de type classe mère.

explicite

```
// Affectation d'une Voiture à un Vehicule
Vehicule ve2= (Vehicule) new Voiture("Dacia",
                                     "Blanc", "Stepway", 4);
ve2.afficher();
//ve2.getModele(); // Pas possible
```

- Je suis un véhicule de marque: Dacia, de couleur: Blanc, ma vitesse est : 0.0 Km/h et je suis en état de: arrêt
- Et je suis aussi une voiture de modèle: Stepway et j'ai 4 portes.

implicite

```
// Affectation d'une Voiture à un Vehicule
Vehicule ve2=new Voiture("Dacia", "Blanc",
                          "Stepway", 4);
ve2.afficher();
//ve2.getModele(); // Pas possible
```

- Je suis un véhicule de marque: Dacia, de couleur: Blanc, ma vitesse est : 0.0 Km/h et je suis en état de: arrêt
- Et je suis aussi une voiture de modèle: Stepway et j'ai 4 portes.

III. CONCEPTS DE LA POO EN JAVA

Transtypage: Références d'objets

Downcasting

Convertir une référence « surclassée » pour « libérer » certaines fonctionnalités cachées par le surclassement.

explicite

```
// Affectation d'un Vehicule à une Voiture
Vehicule ve2= new Voiture("Dacia",
                          "Blanc","Stepway",4);
Voiture vo3= (Voiture)ve2;
System.out.println(vo3.getModele());
```

Stepway

implicite

```
// Affectation d'un Vehicule à une Voiture
Vehicule ve2= new Voiture("Dacia",
                          "Blanc","Stepway",4);
Voiture vo3= ve2; // Erreur compilation
vo3.afficher();
```

Ce type de casting est interdit en Java

III. CONCEPTS DE LA POO EN JAVA

Transtypage: Résumé

- En java, le upcasting est implicite et constitue la base du polymorphisme.
- Un objet d'une classe peut être converti en un objet d'une autre classe si les deux classes ont un lien d'héritage.
- La conversion d'un objet dans le sens fille → mère (**Upcasting**) est **implicite**.
- Par contre, la conversion dans le sens mère → fille (**Downcasting**) **doit être explicite** et **n'est pas toujours possible**. En effet, le downcasting ne permet pas de convertir une instance d'une classe de base en une instance d'une sous-classe. **Exemple:**

```
Voiture vo4= (Voiture)  
              new Vehicule("Dacia", "Blanc");  
// RunTime Error  
vo4.afficher();
```

```
Vehicule ve1= new Vehicule("Dacia", "Blanc");  
Voiture vo4=(Voiture)ve1; // RunTime Error  
vo4.afficher();
```

III. CONCEPTS DE LA POO EN JAVA

Transtypage: Exercice

- Nous avons les classes suivantes:

```
public class Vehicule {  
    .....  
    public Vehicule  
        (String marque, String couleur){  
        .....  
    }  
  
    public void demarrer() {  
        .....  
    }  
  
    public void afficher() {  
        .....  
    }  
}
```

```
public class Voiture extends Vehicule{  
    .....  
    public Voiture(String marque, String couleur,  
        String modele, int nbPortes){  
        .....  
    }  
  
    public String getModele() {  
        .....  
    }  
  
    public void afficher() {  
        .....  
    }  
}
```

III. CONCEPTS DE LA POO EN JAVA

- Quelles sont les erreurs générées par le code suivant?
- Pour chaque méthode ***afficher()***, indiquez le type affiché.

```
Vehicule ve1=new Vehicule("Dacia", "Blanc");  
ve1.demarrer();  
ve1.getModele();  
ve1.afficher();
```

```
Voiture vo1=new Voiture("Renault", "Noir", "Clio 3", 4);  
vo1.demarrer();  
vo1.getModele();  
vo1.afficher();
```

```
Vehicule ve2=new Voiture("Renault", "Blanc", "Clio Campus", 4);  
ve2.demarrer();  
ve2.getModele();  
ve2.afficher();
```

```
Vehicule ve3=new Voiture("Renault", "Blanc", "Clio Campus", 4);  
Voiture vo2=(Voiture) ve3;  
vo2.demarrer();  
vo2.getModele();  
vo2.afficher();
```

III. CONCEPTS DE LA POO EN JAVA

- Quelles sont les erreurs générées par le code suivant?

- Pour chaque méthode *afficher()*, indiquez le type affiché.

```
Vehicule ve1=new Vehicule("Dacia", "Blanc");  
ve1.demarrer();  
ve1.getModele(); // Erreur de compilation  
ve1.afficher(); // Affichage Vehicule
```

```
Voiture vo1=new Voiture("Renault", "Noir", "Clio 3", 4);  
vo1.demarrer();  
vo1.getModele();  
vo1.afficher(); // Affichage Voiture
```

```
Vehicule ve2=new Voiture("Renault", "Blanc", "Clio Campus", 4);  
ve2.demarrer();  
ve2.getModele(); // Erreur de compilation  
ve2.afficher(); // Affichage Voiture
```

```
Vehicule ve3=new Voiture("Renault", "Blanc", "Clio Campus", 4);  
Voiture vo2=(Voiture) ve3;  
vo2.demarrer();  
vo2.getModele();  
vo2.afficher(); // Affichage Voiture
```

III. CONCEPTS DE LA POO EN JAVA

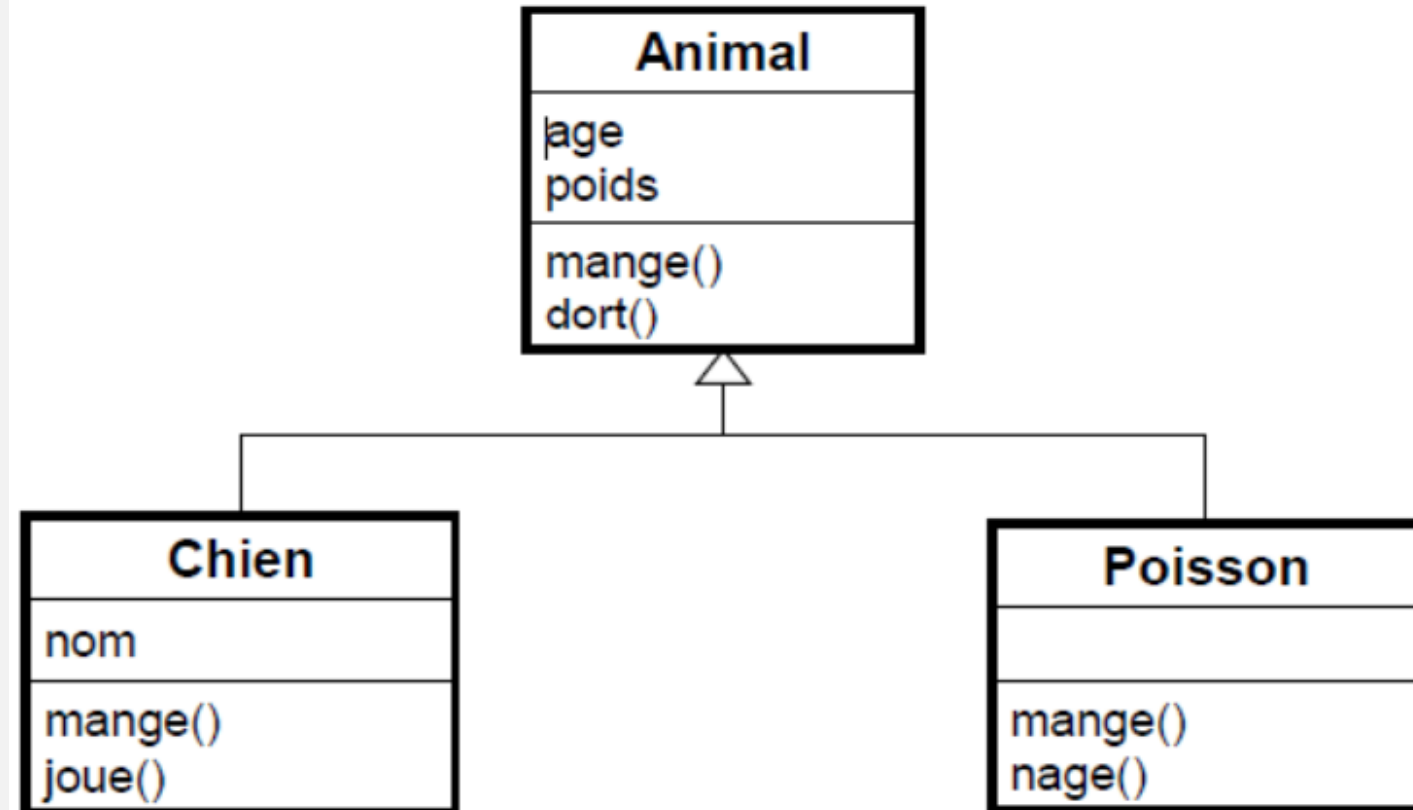
Le mots clé *final*

- Appliqué à **un attribut de type primitif**, ce dernier devient une constante (Impossible de modifier sa valeur).
- Appliqué à un **attribut de type objet** : c'est la référence vers l'objet qui devient constante et non sa valeur. Ce dernier point s'applique également aux tableaux qui sont aussi des références.
- Les **classes** final ne peuvent être dérivées.
- Une **méthode** indiquée comme final ne peut être redéfinie dans une classe dérivée.
- Appliqué à un **paramètre** d'une méthode dont on est sûr qu'elle ne va pas changer de référence.
- Appliqué à des **variables locales** qu'on est sûr que la variable ne va pas changer.

III. CONCEPTS DE LA POO EN JAVA

Exercice

1. Expliquez le diagramme de classe suivant.
2. Si les 3 classes sont implémentées, donnez des exemples de:
 1. Création d'objet Animal et les méthodes accessibles.
 2. Création d'objet Chien et les méthodes accessibles.
 3. Création d'objet Poisson et les méthodes accessibles.
 4. Upcasting et les méthodes accessibles.
 5. Downcasting et les méthodes accessibles.



III. CONCEPTS DE LA POO EN JAVA

Exercice

- Les classes « Chien » et « Poisson » sont des sous classes de « Animal ».
- « Animal » est la superclasse de « Chien » et de « Poisson ».
- Les classes « Chien » et « Poisson » héritent des attributs membres et méthodes de « animal » (age, poids, dort()).
- Les classes « Chien » et « Poisson » redéfinissent (modifient le comportement de) la méthode mange().
- La classe « Chien » ajoute le champ nom et la méthode joue().
- La classe « Poisson » ajoute la méthode nage().

III. CONCEPTS DE LA POO EN JAVA

Exercice

- Si l'on implémente les classes « Animal », « Chien » et « Poisson » et que l'on écrit les instructions suivantes :

Chien pitbull= new Chien(...);

pitbull.mange();

- La méthode mange() qui sera invoquée est celle qui est définie dans la classe fille « Chien » (car pitbull est du type « Chien » et la méthode mange() est redéfinie pour cette classe fille).
- Si la classe fille « Chien » n'a pas redéfini la méthode mange(), alors la méthode mange() de « Animal » qui sera invoquée (la classe Chien en hériterait).

III. CONCEPTS DE LA POO EN JAVA

Recherche dynamique de méthode

- Dans de nombreuses situations, en java, où on a des **relations d'héritage**, la détermination de la méthode à appeler **n'est pas réalisée lors de la compilation**.
- **Seulement à l'exécution** la machine virtuelle détermine la méthode à appeler selon le type effectif de l'objet référencé à ce moment là.
- Ce mécanisme est appelé «**Recherche dynamique de méthode** » . Il constitue la base de la mise en œuvre du polymorphisme.
- Il s'agit donc d'une aptitude d'adaptation dynamique de comportement selon les objets en présence.
- Avec l'encapsulation et l'héritage, le polymorphisme est une des propriétés essentielles de la programmation orientée objet.

III. CONCEPTS DE LA POO EN JAVA

Exemple polymorphisme

- L'exemple ci-dessous permet de montrer le principe du polymorphisme. Il est basé sur les classes qu'on a défini précédemment (« Animal », « Chien » et « Poisson »).
- Si l'on veut enregistrer et manipuler une collection d'animaux « une ménagerie », Alors on peut créer et remplir le tableau suivant :

```
// Création du tableau
Animal[] menagerie = new Animal[6];
// Remplissage du tableau
menagerie[0] = new Poisson(...);
menagerie[1] = new Chien(...);
menagerie[2] = new Chien(...);
menagerie[3] = new Animal(...);
menagerie[4] = new Poisson(...);
menagerie[5] = new Chien(...);
```

III. CONCEPTS DE LA POO EN JAVA

Exemple polymorphisme

- Le polymorphisme, dans ce cas, nous permet d'écrire une méthode **nourrir ()** dont la fonction est de donner à manger à chaque «animal» contenu dans le tableau passé en paramètre en invoquant successivement la méthode `mange()` pour chacun d'eux.
- Appel à la méthode **mange()** pour chaque « animal » contenu dans le tableau passé en paramètre

```
public static void nourrir(Animal[] tabAnimaux) {  
    for (int i=0; i<tabAnimaux.length; i++) {  
        if (tabAnimaux[i] != null) {  
            tabAnimaux[i].mange(); }  
        }  
    }
```

III. CONCEPTS DE LA POO EN JAVA

Exemple polymorphisme

- Il est possible ensuite d'invoquer la méthode **nourrir()** en lui passant en paramètre la ménagerie précédemment créée : **nourrir (menagerie) ;**
- Sur la base du contenu de menagerie, la méthode nourrir() appellera successivement :
 - mange() de la classe Poisson**
 - mange() de la classe Chien**
 - mange() de la classe Chien**
 - mange() de la classe Animal**
 - mange() de la classe Poisson**
 - mange() de la classe Chien**
- La méthode nourrir() n'a pas besoin de déterminer elle-même quelle méthode doit être invoquée pour chaque « Animal ».
- Le polymorphisme va faire en sorte que le message mange() soit interprété (à l'exécution) de manière appropriée selon les objets qui le reçoivent.

III. CONCEPTS DE LA POO EN JAVA

Classe Object

- En Java, chaque classe que l'on crée possède une classe parente.
- Toutes les classes définies en JAVA sont des sous-classes de la classe de base « Object ».
- Ceci est automatique et n'a pas besoin d'être spécifié.
- En effet, si on ne définit pas explicitement une classe parente, avec le mot clé extends, la classe mère par défaut est « Object ». Elle est définie dans le package java.lang.
- la classe « Object » est la seule classe Java qui ne possède pas de classe mère.
- Il existe plusieurs méthodes dans la classe « Object ». Les classes définies par un programmeur hériteront toutes ces méthodes. Nous en verrons **cinq**.

III. CONCEPTS DE LA POO EN JAVA

Méthode	Fonctionnalité
String toString()	Retourne une chaîne de caractères représentant l'objet.
boolean equals(Object obj)	Indique si l'objet est égal à un autre.
Class getClass()	Retourne la classe d'exécution d'un objet. Par exemple : Personne m=new Personne(« Enseignant",« Etudiant"); System.out.println("Classe de Personne : " + m.getClass()) ; //affiche Class Personne . System.out.println(m.getClass().getName()) ;//affiche le nom de la classe, ici Personne.
protected void finalize()	Appelée par le « garbage collector » lorsqu'il n'y a plus de références à l'objet.
protected Object clone()	Permet de créer et de retourner une copie de l'objet.